In-Class Project: Fixed-Wing Aircraft Pitch Controller

1 Lab Overview

Previously in class, we learned about aircraft controls including a pitch control system. For this lab, we shall build a small control system simulation evaluating the P, PI, and PID controllers and a simple physics model.

In this lab, you will learn the mathematical logic that governs the plant model and control system. You will model the pitch response of a fixed-wing aircraft and design a basic feedback controller to maintain a specific pitch angle (θ).

The Scenario: A pitch-hold autopilot shall be implemented for a simple fixed-wing aircraft by control of the elevator control surface. If a pilot wants to pull the nose up by exactly 1 degree (1.0 unit), the control system should translate the command change of the elevator to achieve the desired pitch. You will first see how the natural response of the plant to elevator commands, and then how a control system can provide a smoother increase in pitch with a lower settle time.

---

2 From Physics to Math: Understanding the Aircraft

Before we write any code, we need to understand how the airplane physically behaves.

2.1 Time Domain Model

We use Newton's laws of motion to write a differential equation that describes the pitch of the aircraft over time (t).

$$I_y \frac{d^2\theta}{dt^2} = \sum(M_{aero})$$

(net torque on rotating body) = (Sum of Aerodynamic Pitching Moments (torques) acting upon the aircraft and pilot command moment.)

$$I_y \frac{d^2\theta}{dt^2} = \underbrace{-C_\alpha \theta(t) - C_q \frac{d\theta}{dt}}_{\text{Airplane resisting}} + \underbrace{C_\delta \delta_e(t) + C_{\dot{\delta}} \frac{d\delta_e}{dt}}_{\text{Pilot commanding}}$$

For the general aviation aircraft in this lab, we will do some simple arithmetic to move the negative terms of the right-hand side to the left hand side and divide both sides by I_y.

$$\frac{d^2\theta}{dt^2} + 0.739\frac{d\theta}{dt} + 0.921\theta(t) = 1.15\frac{d\delta_e}{dt} + 0.17\delta_e(t)$$

What does this mean physically?

- **The Left Side (The Airplane):** This is the aircraft's natural resistance to changing direction.
  - $\boldsymbol{\frac{d^2\theta}{dt^2}}$ **(Pitch Acceleration):** The inertia of the heavy aircraft resisting rotation.
  - $\boldsymbol{0.739\frac{d\theta}{dt}}$ **(Aerodynamic Damping):** As the tail moves up or down, the air pushes back against it, acting like a shock absorber.
  - $\boldsymbol{0.921\theta(t)}$ **(Aerodynamic Stiffness):** Like a weathervane, the airplane naturally wants to fly straight into the wind. This is the restoring force.

- **The Right Side (The Pilot):** This is the force the pilot applies using the elevator (δ_e).
  - $\mathbf{0.17\delta_e(t)}$ **(Elevator Position):** The raw aerodynamic force created by deflecting the elevator.
  - $\mathbf{1.15\frac{d\delta_e}{dt}}$ **(Elevator Rate):** The "whip" effect. Yanking the stick back rapidly generates a sudden, extra spike in force.

- The **coefficients** shown include
  - $0.739 = \frac{C_q}{I_y}$ (Aerodynamic Damping divided by Inertia)
  - $0.921 = \frac{C_\alpha}{I_y}$ (Aerodynamic Stiffness divided by Inertia)
  - $1.15 = \frac{C_{\dot{\delta}}}{I_y}$ (Elevator Rate Force divided by Inertia)
  - $0.17 = \frac{C_\delta}{I_y}$ (Elevator Position Force divided by Inertia)

2.2 The s-Domain "Cheat Code"

Solving differential equations with varying pilot inputs in real-time is mathematically brutal. To make this easier for autopilot computers, engineers use the Laplace Transform to move from the Time Domain (t) to the s-Domain.
The rule is simple: Every time you see an s, it means "take the derivative."

- First derivative (Velocity) becomes s.
- Second derivative (Acceleration) becomes s^2.

If we swap the derivatives in our physics equation for s and divide the Output (Pitch, Θ) by the Input (Elevator, Δ_e),

FROM:
$\frac{d^2\theta}{dt^2} + 0.739\frac{d\theta}{dt} + 0.921\theta(t) = 1.15\frac{d\delta_e}{dt} + 0.17\delta_e(t)$

TO:
$(s^2 + 0.739s + 0.921)\,\dot{\Theta}(s) = (1.15s + 0.17)\,\Delta_e(s)$

Solving for $G(s) = \frac{\Theta(s)}{\Delta_e(s)}$, we get our Transfer Function, $G(s)$

$G(s) = \frac{\Theta(s)}{\Delta_e(s)} = \frac{1.15s+0.17}{s^2+0.739s+0.921}$

Notice the denominator: $s^2 + 0.739s + 0.921$. The middle number tells us the damping ratio is about $\zeta \approx 0.37$ (i.e. 1/2 the middle term). This means the airplane is underdamped. It won't smoothly snap to a new pitch; it will wobble naturally.

The Control Loop

To control this wobbling airplane, we will use a Feedback Loop.

1. The Reference: The target pitch (1.0).
2. The Plant $G(s)$: The mathematical model of our airplane.
3. The Controller $C(s)$: The autopilot logic you will design to calculate the Error (Target minus Actual Pitch) and move the elevator accordingly.

3 Implementation Instructions

The following instructions are for Matlab. Alternative instructions are provided at the end of the lab instructions.
Requirement: Control System Toolbox

3.1 Task 1: Open-Loop Response (No Autopilot)

First, we will see what happens if we just deflect the elevator without an autopilot actively managing the error.

```
% Task 1 – Matlab Code


% 1. Define the transfer function G(s)
num = [1.15, 0.17];        % Numerator coefficients
den = [1, 0.739, 0.921];   % Denominator coefficients
G = tf(num, den);          % Creates the mathematical 'Plant'

% 2. Plot the step response
figure;                    % Opens a new blank plot window
step(G);                   % Simulates a sudden 1-unit input to the elevator
title('Task 1: Open-Loop Pitch Response');
```

Reflections: *Take a moment to learn more about the Matlab methods used including tf, figure, step, and title. To do this, type* ***help*** *followed by the function you want to learn more about in the Matlab prompt, e.g. "help tf". In the Matlab terminal, the help guide is output, which provides detailed information on usage.*

Observe the final value. Does the nose actually reach the 1.0 target? How long until it settles? What is the peak value?

Note:

3.2 Task 2: Proportional Control (K_p)

Now we add a controller that multiplies the error by a Proportional Gain (K_p) to push the aircraft harder.

```
% Task 2 – Matlab Code


% 1. Define the Proportional controller
Kp = 2;
Cp = pid(Kp);    % Creates a P-controller with a gain of 2

% 2. Connect the controller and plant in a feedback loop
%    The '1' tells MATLAB to use standard negative feedback
%   (comparing output to input)
Tp = feedback(Cp * G, 1);

% 3. Plot the closed-loop step response
figure;
step(Tp);
title('Task 2: Closed-Loop P-Control Response');
```

Observe the new steady-state value. Is there still an offset (error) from the 1.0 target?

Notes:

3.3 Task 3: PI Control (K_p, K_i)

To eliminate the remaining error, we add an Integral term (K_i) which accumulates past error and keeps pushing the elevator until the error is exactly zero.

The feedback loop in the S-domain where two systems controller and plant are interacting, the forward path becomes: Forward Path $= C_p(s) \cdot G(s)$ (error is passed into the control which sends signals to the plan to which the error can be measured and fed back into the forward path for the next step).

```
% Task 3 – Matlab Code

% 1. Define the Proportional-Integral controller
Ki = 0.5;
Cpi = pid(Kp, Ki);   %Creates a PI-controller with both gains

% 2. Close the loop
Tpi = feedback(Cpi * G, 1);

% 3. Plot the final response
figure;
step(Tpi);
title('Task 3: Closed-Loop PI-Control Response');
```

Observe the final value now. Note the initial response and the gradual increase to at/near 1.

Notes:

3.4 Task 4: PID Control (K_p, K_i, K_d)

In a real aircraft, pitching up that far past the target before aggressively pitching down would subject passengers to severe negative G-forces.

Your Task: Upgrade your PI controller to a PID controller. A Derivative (K_d) term looks at the *rate of change* and "applies the brakes" as the nose approaches the target.

1. Keep your previous values ($K_p =, K_i = 0.$) and add a derivative gain of $K_d = 1.5$.
2. Adjust K_d up or down until the jitter is reduced but the system still reaches the target reasonably fast.
3. Include your plot in your report

MATLAB Implementation for PID Challenge:

```
% Task 4 – Matlab Code
% 1. Define the full PID controller
Kd = 1.5;              % Derivative gain (the "brakes")
C_pid = pid(Kp, Ki, Kd); % Creates the PID controller

% 2. Close the loop
T_pid = feedback(C_pid * G, 1);

% 3. Plot the PI and PID responses together to see the improvement
figure;
step(Tpi, 'b');        % Plots the old PI response in Blue
hold on;               % Keeps the first line on the graph
step(T_pid, 'r');      % Plots the new PID response in Red
legend('PI Control ', 'PID Control');
title(Task 4: PID vs PI Control');
hold off;
```

Notes:

4 Lab Report Requirements

Your report (3–5 pages) must include:

1. Introduction: Briefly explain the aerodynamic concepts of damping and restoring force discussed in Section 2.

2. System Plots: Include labeled plots for Tasks 1 through 4.

3. Data Table: Fill out the following for your report:

Metric	**Open-Loop**	**P-Control** (Kp=2)	**PI-Control** (Kp=2,Ki=0.5)	**PID-Control** (Kp=2,Ki=0.5, $\boldsymbol{Kd = 1.5}$)
Final Value (Steady State)				
Steady State Error				
Peak value				

4. Analysis Question: Explain why K_p alone wasn't enough to reach the target and how the Integral (K_i) term fixed the error.

5. Analysis Question: Explain how K_i impacted your controller response. Describe the changes in the plot? Did the dampening coefficient reduce error? Did it impact settle time?

6. Analysis Question: Explain how K_d impacted your controller response. Describe the changes in the plot? Did the dampening coefficient reduce error? Did it impact settle time?

7. Analysis Question: Through tuning your coefficients, were you able to improve the accuracy or settle time of the response? What were the coefficient values? What was the observed settle time (best estimate)?

8. What are some important lessons learned / insights gained from this activity?

5 Grading Rubric (100 Points)

Category	**Description**	**Points**
Code Execution	Transfer function and feedback loops implemented correctly.	30
Data Accuracy	Table values match simulation results accurately.	20

Analysis	Clear explanation on the relationship between the coefficients and the model's performance.	40
Formatting	Professional tone, labeled axes, and clear legends.	10

6 Python Instructions

Requirement: pip install control matplotlib numpy

- **Task 1: Open-Loop Response (No Autopilot)**
  1. Define the system:
     G = ct.TransferFunction([1.15, 0.17], [1, 0.739, 0.921])
  2. Simulate: t,
     y = ct.step_response(G); plt.plot(t, y); plt.show()

- **Task 2: Proportional Control (K_p)**
  1. Apply a gain of 2:
     forward_path = 2 * G
  2. Close the loop:
     Tp = ct.feedback(forward_path, 1)
  3. Plot using the same step_response method.

- **Task 3: PI Control (K_p, K_i)**
  1. Define the PI controller:
     Cpi = ct.TransferFunction([2, 0.5], [1, 0])
  2. Combine and close loop:
     Tpi = ct.feedback(Cpi * G, 1)
  3. Plot and observe if the steady-state error is eliminated.

- **Task 4: PID Control (K_p, K_i, K_d)**
  - *C_pid = ct.TransferFunction([1.5, 2, 0.5], [1, 0]))*